

UC01–UC06 三层模型源

观澜视频平台 · 第 4 组 · 2026 夏季软件工程基础实践

可编辑源文件与本 PDF 同时提交；本文由仓库脚本生成，页脚标注页码，最终以 Git 中的 Markdown/Mermaid 源为准。

UC01–UC05 三层模型源文件

■ Task：DOC-01；适用范围：UC01–UC05。UC06 的系统级、组件级和对象级状态模型见 10-uc06-state-diagrams.md。

■ 本文件使用 Mermaid 保存可版本化的模型源。模型描述的是已合并的 Gateway + 四服务边界；最终行为由 REG-01 的单体/Gateway 12/12 回归和对应 contract 测试验证。

1. UC01 账户访问与资料维护

SYS-SEQ01 系统级交互

代码 / 模型源 (mermaid)

```
sequenceDiagram
    actor User as 用户
    participant Web as Vue Web
    participant Gateway as API Gateway
    participant Identity as identity-community
    participant DB as Identity MySQL

    User->>Web: 注册/登录/修改资料/关注
    Web->>Gateway: Bearer Token + requestId
    Gateway->>Identity: 转发公开请求
    Identity->>DB: 事务写入 User/Profile/Follow
    DB-->>Identity: 持久化结果
    Identity-->>Gateway: 统一响应 + requestId
    Gateway-->>Web: 200/201 或可解释 4xx
    Web-->>User: 更新会话与页面状态
```

COMP-SEQ01 组件级职责

代码 / 模型源 (mermaid)

```
flowchart LR
    UI[登录/资料/关注页面] --> GW[Gateway 路由与认证]
    GW --> AUTH[Auth Controller]
    GW --> PROFILE[Profile Controller]
    GW --> SOCIAL[Follow / Dynamic Controller]
    AUTH --> SESSION[Session Nonce 与 Token]
    PROFILE --> REPO[Identity Prisma Repository]
    SOCIAL --> REPO
    SESSION --> REPO
    REPO --> DB[(Identity schema)]
    AUTH -. 失败回滚 .-> MONO[Monolith fallback]
```

OBJ-SEQ01 对象级模型

代码 / 模型源 (mermaid)

```
classDiagram
    class User {
        +String id
        +String username
        +String role
        +String sessionNonce
    }
    class Profile {
        +String userId
        +String nickname
        +String avatar
    }
    class Follow {
        +String followerId
        +String followingId
    }
    class DynamicPost {
        +String authorId
    }
```

```

+String content
}
class Notification {
+String receiverId
+String requestId
+String type
}
User "1" *-- "0..1" Profile
User "1" --> "0..*" Follow : follower/following
User "1" --> "0..*" DynamicPost : authors
User "1" --> "0..*" Notification : receives

```

2. UC02 视频发现、播放与观看记录

SYS-SEQ02 系统级交互

代码 / 模型源 (mermaid)

```

sequenceDiagram
    actor Viewer as 观众
    participant Web as Vue Web
    participant Gateway as API Gateway
    participant Content as content-media
    participant Identity as identity-community
    participant MinIO as MinIO
    participant DB as Content MySQL

    Viewer->>Web: 浏览推荐/搜索/视频详情
    Web->>Gateway: GET feed/search/videos
    Gateway->>Content: 已开放读能力
    Content->>DB: 查询视频、分类、计数与进度
    Content->>Identity: 批量查询作者摘要
    Content-->>Web: 视频投影与媒体 URL
    Web->>Gateway: Range 媒体请求
    Gateway->>Content: media proxy
    Content-->>MinIO: Range GET
    MinIO-->>Viewer: 206/200 媒体流
    Viewer->>Gateway: 播放/观看进度 requestId
    Gateway->>Content: 可信用户上下文
    Content->>DB: 幂等写入 Play/WatchProgress

```

COMP-SEQ02 组件级职责

代码 / 模型源 (mermaid)

```

flowchart LR
    HOME[首页/搜索/详情] --> GW[Gateway capability map]
    GW --> DISCOVERY[Recommend/Search Service]
    GW --> VIDEO[Video Query Service]
    GW --> MEDIA[Media Proxy]
    GW --> WATCH[Play/Progress Service]
    DISCOVERY --> REPO[Content Repository]
    VIDEO --> REPO
    WATCH --> RECEIPT[Write Receipt]
    RECEIPT --> REPO
    VIDEO --> SUMMARY[Identity Batch Summary]
    MEDIA --> OBJ[(MinIO objects)]
    REPO --> DB[(Content schema)]

```

OBJ-SEQ02 对象级模型

代码 / 模型源 (mermaid)

```

classDiagram
    class Video {
+String id
+String creatorId
+String status
+Int playCount
    }
    class VideoAsset {
+String objectKey
+String mimeType
+String videoid
    }

```

```

class Category {
+Int id
+String name
}
class WatchProgress {
+String userId
+String videoid
+Int position
+Boolean completed
}
class PlayRecord {
+String requestId
+String userId
+String videoid
}
Video "1" o-- "0..*" VideoAsset
Video "0..*" -- "0..*" Category
Video "1" --> "0..*" WatchProgress
Video "1" --> "0..*" PlayRecord

```

3. UC03 投稿、审核、发布与重提

SYS-SEQ03 系统级交互

代码 / 模型源 (mermaid)

```

sequenceDiagram
    actor Creator as 创作者
    actor Admin as 审核员
    participant Web as Vue Web/Admin
    participant Gateway as API Gateway
    participant Content as content-media
    participant Governance as governance-ai
    participant MinIO as MinIO

    Creator->>Web: 上传媒体并保存草稿
    Web->>Gateway: multipart + requestId
    Gateway->>Content: 可信 creator 上下文
    Content->>MinIO: 上传原始对象
    Content->>Content: ffprobe/Range/资产与稿件事务
    Creator->>Gateway: 提交审核
    Gateway->>Content: SUBMIT 状态转换
    Content->>Governance: service JWT + requestId
    Governance-->>Admin: 待审快照
    Admin->>Governance: 通过或驳回
    Governance->>Content: 应用审核决定
    Content-->>Creator: PUBLISHED 或 REJECTED
    Creator->>Content: 修改后重提/撤回

```

COMP-SEQ03 组件级职责

代码 / 模型源 (mermaid)

```

flowchart LR
    CREATOR[创作中心] --> GW[Gateway 身份与路由]
    GW --> UPLOAD[Upload]
    GW --> SUBMIT[Submission Service]
    UPLOAD --> PROBE[ffprobe / MIME Guard]
    UPLOAD --> MINIO[MinIO]
    UPLOAD --> DB[(Content schema)]
    SUBMIT --> RECEIPT[ContentWriteReceipt]
    SUBMIT --> GOVCLIENT[Governance Client]
    GOVCLIENT --> REVIEW[Review Queue / Decision]
    REVIEW --> APPLY[Content Decision Endpoint]
    APPLY --> DB
    SUBMIT -.timeout/失败.-> RETRY[保留状态并可重试]

```

OBJ-SEQ03 对象级模型

代码 / 模型源 (mermaid)

```

classDiagram
    class VideoAsset {
+String uploaderId
+String objectKey

```

```

+String mimeType
+String videoid
}
class Video {
+String id
+String creatorId
+String status
+String rejectReason
+DateTime publishedAt
}
class VideoReview {
+String contentVideoid
+String status
+String requestId
}
class ModerationDecision {
+String action
+String reason
+DateTime decidedAt
}
class ContentWriteReceipt {
+String requestId
+String operation
+String payloadHash
}
}
VideoAsset "0..*" --> "0..1" Video
Video "1" --> "0..*" VideoReview
VideoReview "1" --> "0..1" ModerationDecision
Video "1" --> "0..*" ContentWriteReceipt

```

4. UC04 互动、通知与幂等

SYS-SEQ04 系统级交互

代码 / 模型源 (mermaid)

```

sequenceDiagram
    actor Actor as 互动用户
    actor Creator as 内容作者
    participant Web as Vue Web
    participant Gateway as API Gateway
    participant Content as content-media
    participant Identity as identity-community
    participant DB as Content MySQL

    Actor->>Web: 评论/回复/点赞/收藏/弹幕
    Web->>Gateway: requestId + Bearer Token
    Gateway->>Content: gateway.user.forward JWT
    Content->>DB: 主体、计数、Receipt、Outbox 同事务
    DB-->>Content: COMMIT
    Content-->>Actor: 成功或 409 payload conflict
    Content->>Identity: Outbox 投递通知 ( 有界重试 )
    Identity-->>Creator: COMMENT/REPLY/LIKE/FAVORITE 通知
    Note over Content,Identity: 通知失败不回滚已提交的互动主体

```

COMP-SEQ04 组件级职责

代码 / 模型源 (mermaid)

```

flowchart LR
    UI[互动控件] --> GW[Gateway trusted context]
    GW --> COMMENT[Comment Service]
    GW --> LIKE[Like Service]
    GW --> FAVORITE[Favorite Service]
    GW --> DANMAKU[Danmaku Service]
    COMMENT --> TX[Prisma Transaction]
    LIKE --> TX
    FAVORITE --> TX
    DANMAKU --> TX
    TX --> RECEIPT[ContentWriteReceipt]
    TX --> OUTBOX[NotificationOutbox]
    OUTBOX --> NOTIFY[Identity Notification API]
    OUTBOX -.timeout.-> RETRY[指数退避, 最多 5 次]

```

OBJ-SEQ04 对象级模型

代码 / 模型源 (mermaid)

```
classDiagram
class Comment {
+String id
+String videoid
+String userId
+String parentId
+String status
}
class VideoLike {
+String videoid
+String userId
}
class FavoriteItem {
+String folderId
+String videoid
}
class Danmaku {
+String videoid
+String userId
+Int offsetMs
}
class ContentWriteReceipt {
+String requestId
+String payloadHash
}
class NotificationOutbox {
+String requestId
+String status
+Int attempts
}
ContentWriteReceipt "1" --> "1" Comment : protects
ContentWriteReceipt "1" --> "0..1" VideoLike : protects
ContentWriteReceipt "1" --> "0..1" FavoriteItem : protects
ContentWriteReceipt "1" --> "0..1" Danmaku : protects
Comment "1" --> "0..1" NotificationOutbox
VideoLike "1" --> "0..1" NotificationOutbox
FavoriteItem "1" --> "0..1" NotificationOutbox
```

5. UC05 直播、录播与币账本

SYS-SEQ05 系统级交互

代码 / 模型源 (mermaid)

```
sequenceDiagram
actor Host as 主播
actor Viewer as 观众
participant Web as Vue Web
participant Gateway as API Gateway
participant Live as live-reward
participant SRS as SRS
participant Content as content-media
participant DB as Live MySQL

Host->>Gateway: 创建房间并开播
Gateway->>Live: 可信身份 + requestId
Live->>SRS: readiness/API probe
Live->>DB: LiveRoom/LiveSession 持久化
Viewer->>Gateway: 加入、消息、兼容帧、投币
Gateway->>Live: SSE/HTTP 与账本请求
Live->>DB: ViewerEvent/Message/CoinTransaction
Host->>Gateway: 停播并登记录播
Live->>Content: service JWT + REPLAY asset
Content-->>Live: contentVideoid 或可重试失败
Live->>DB: Session=ENDED 与 ReplayRegistration
```

COMP-SEQ05 组件级职责

代码 / 模型源 (mermaid)

```
flowchart LR
LIVEUI[直播中心/房间] --> GW[Gateway UC05 capability]
```

```

GW --> ROOM[Room Service]
GW --> SESSION[Session Service]
GW --> MESSAGE[Message/SSE Service]
GW --> VIEWER[Viewer/Frame Service]
GW --> LEDGER[Coin Ledger]
SESSION --> SRS[SRS Client]
ROOM --> REPO[Live Prisma Store]
SESSION --> REPO
MESSAGE --> REPO
VIEWER --> REPO
LEDGER --> REPO
SESSION --> REPLAY[Content Replay Client]
REPLAY -.timeout/5xx.-> RETRY[FAILED_RETRYABLE]
REPO --> DB[(Live schema)]

```

OBJ-SEQ05 对象级模型

代码 / 模型源 (mermaid)

```

classDiagram
    class LiveRoom {
        +String id
        +String ownerId
        +String status
    }
    class LiveSession {
        +String id
        +String roomId
        +String status
        +String replayVideoid
    }
    class LiveMessage {
        +String sessionId
        +String senderId
        +String type
    }
    class ViewerEvent {
        +String sessionId
        +String viewerId
        +String action
    }
    class CoinAccount {
        +String userId
        +Int balance
    }
    class CoinTransaction {
        +String requestId
        +String fromUserId
        +String toUserId
        +Int amount
    }
    class ReplayRegistration {
        +String sessionId
        +String status
        +String contentVideoid
    }
    LiveRoom "1" --> "0..*" LiveSession
    LiveSession "1" --> "0..*" LiveMessage
    LiveSession "1" --> "0..*" ViewerEvent
    LiveSession "1" --> "0..1" ReplayRegistration
    CoinAccount "1" --> "0..*" CoinTransaction

```

6. 追溯与验证

UC	代码边界	自动验证	最终结果
UC01	Gateway + identity-community	identity contract、REG-01 UC01 (单体/Gateway)	PASS / PASS
UC02	Gateway + content-media + MinIO	content contract、REG-01 UC02 (单体/Gateway)	PASS / PASS
UC03	content-media + governance-ai	publishing/review contract、REG-01 UC03 (单体/Gateway)	PASS / PASS
UC04	content-media + identity-community	receipt/outbox/notification contract、REG-01 UC04 (单体/Gateway)	PASS / PASS
UC05	live-reward + content-media + SRS	live/replay/ledger contract、REG-01 UC05 (单体/Gateway)	PASS / PASS

详细 endpoint 清单、真实运行值和对应 PR 见 [06-evidence-and-dod.md](#)、[00-progress.md](#) 与 REG-01 JSON 输出。

UC06 举报、审核与治理处置：三层状态图

- 需求：REQ06；用例：UC06；模型：SYS-STATE06、COMP-STATE06、OBJ-STATE06。
- 追溯测试：UNIT-TC06、INT-TC06、E2E-TC06。原有 SYS-SEQ06、COMP-SEQ06、OBJ-SEQ06 保留给后续三层序列图，不与本状态图编号混用。

1. 建模口径

- 系统级状态图描述一次举报治理业务从提交到结果可验证的整体状态。
- 组件级状态图描述前端、report、admin、可选 AI 和内容处置组件之间的状态交接。
- 对象级状态图描述 ReportRecord 与目标视频、评论或弹幕形成的联合对象状态。
- 实线状态包含现有实现和本用例要求的目标设计；带“待实现”或“设计瞬态”的节点不得当作当前代码已经具备的能力。
- KEEP 表示举报不成立：举报记录进入 REJECTED，视频保持原状态，评论或弹幕按当前实现恢复为 NORMAL。
- HIDE / DELETE 表示举报成立：举报记录进入 PROCESSED；视频两种动作当前都映射为 REJECTED，评论或弹幕分别进入 HIDDEN / DELETED。

2. SYS-STATE06 系统级状态图

代码 / 模型源 (mermaid)

```
stateDiagram-v2
    direction TB

    [*] --> Ready
    state "待提交举报" as Ready
    state "校验举报" as Validating
    state "提交被拒绝" as SubmissionRejected
    state "待管理员审核" as PendingReview
    state "管理员审核中" as Reviewing
    state "AI 辅助 ( 可选 )" as AiAssist
    state "应用处置" as Applying
    state "举报已驳回<br/>内容保留" as Rejected
    state "举报已处理<br/>内容隐藏/下架" as Processed
    state "处置失败" as Failed

    Ready --> Validating : 用户提交举报
    Validating --> SubmissionRejected : 未登录 / 目标不存在
    Validating --> PendingReview : 校验通过或复用重复举报

    PendingReview --> PendingReview : 普通用户调用管理员接口 / 401，状态不变
    PendingReview --> Reviewing : 管理员查看举报
    Reviewing --> AiAssist : 请求辅助建议
    AiAssist --> Reviewing : 返回建议或降级人工
    Reviewing --> Rejected : 驳回举报 / KEEP
    Reviewing --> Applying : 通过举报 / HIDE 或 DELETE
    Applying --> Processed : 目标与举报记录更新成功
    Applying --> Failed : 更新失败
    Failed --> Applying : 重试
    Failed --> PendingReview : 回滚

    SubmissionRejected --> [*]
    Rejected --> [*]
    Processed --> [*]
```

系统级只保留业务主状态。AI 不可用时回到人工审核；处置失败时可以重试或回滚到待审核。

3. COMP-STATE06 组件级状态图

代码 / 模型源 (mermaid)


```

stateDiagram-v2
    direction TB

    [*] --> UserSubmit
    state "前端<br/>提交举报" as UserSubmit
    state "report<br/>校验并保存" as ReportAccept
    state "请求被拒绝" as RequestRejected
    state "admin<br/>审核举报" as AdminReview
    state "AI<br/>辅助建议 ( 可选 )" as AiAssist
    state "admin<br/>形成决定" as AdminDecision
    state "内容组件<br/>更新目标" as ContentUpdate
    state "admin<br/>记录结果与审计" as RecordResult
    state "处理完成" as Completed

    UserSubmit --> ReportAccept : POST /reports
    ReportAccept --> RequestRejected : 鉴权或校验失败
    ReportAccept --> AdminReview : 保存为 PENDING
    AdminReview --> AdminReview : 非管理员请求 / 401
    AdminReview --> AiAssist : 可选调用
    AiAssist --> AdminDecision : 建议或降级结果
    AdminReview --> AdminDecision : 人工审核
    AdminDecision --> RecordResult : KEEP
    AdminDecision --> ContentUpdate : HIDE / DELETE
    ContentUpdate --> RecordResult : 更新成功
    ContentUpdate --> AdminReview : 失败并回滚
    RecordResult --> Completed

    RequestRejected --> [*]
    Completed --> [*]

```

组件级只表示状态交接，不展开每个组件内部的所有校验步骤。关键边界仍是内容更新与举报审计的一致性。

4. OBJ-STATE06 对象级状态图

代码 / 模型源 (mermaid)

```

stateDiagram-v2
    direction TB

    [*] --> PendingObject
    state "Report=PENDING<br/>目标状态不变" as PendingObject
    state "处置中" as ApplyingObject
    state "Report=REJECTED<br/>目标保留" as RejectedObject
    state "Report=PROCESSED<br/>视频下架" as ProcessedVideo
    state "Report=PROCESSED<br/>评论/弹幕隐藏或删除" as ProcessedText
    state "处置失败<br/>目标回滚" as RetryObject

    PendingObject --> RejectedObject : KEEP
    PendingObject --> ApplyingObject : HIDE / DELETE
    ApplyingObject --> ProcessedVideo : 视频
    ApplyingObject --> ProcessedText : 评论或弹幕
    ApplyingObject --> RetryObject : 更新失败
    RetryObject --> ApplyingObject : 重试
    RetryObject --> PendingObject : 回滚

    RejectedObject --> [*]
    ProcessedVideo --> [*]
    ProcessedText --> [*]

```

对象级保留 **ReportRecord** 与目标内容的联合终态；操作者、时间和原因统一在终态写入，不再拆成额外节点。

5. 与当前代码的对应关系

能力	当前实现	图中位置	结论
举报者鉴权	ReportController 调用 AuthService.requireUser	ReportAccept	已实现
原因与目标校验	DTO 校验原因长度；ReportService 查询视频/评论/弹幕是否存在	ReportAccept	已实现
保存待处理举报	创建 ReportRecord，默认 PENDING	ReportAccept → AdminReview	已实现
管理员权限	AdminController.requireAdmin 检查 ADMIN	AdminReview	已实现
举报列表与目标摘要	GET /admin/reports 关联举报者和目标内容	AdminReview	已实现
保留/隐藏/删除	POST /admin/reports/:id 更新目标及举报记录	AdminDecision、ContentUpdate	已实现基础路径
操作者、时间、原因	写入 handlerId、handledAt、handleNote	RecordResult	已实现，但原因在 DTO 中仍可选
重复举报治理	没有唯一约束或活跃举报查询	ReportAccept	待实现
AI 辅助审核	agent/review-preview 仅返回开发期 mock，未接入举报后台；comment-ai 用于 @grok 评论回复	AiAssist	可选能力，当前未接入
原子处置/回滚	目标与举报记录是两个顺序 update，无事务/补偿	ContentUpdate → RecordResult	关键一致性缺口
防止重复处置	接口未限制仅处理 PENDING 举报	PendingObject	待实现

6. 测试落点

追溯编号	建议验证重点	覆盖的关键转换
UNIT-TC06	举报原因、目标类型与目标存在性；管理员动作映射；仅允许 PENDING 进入处置；重复举报策略	Validating、AdminDecision、PendingObject
INT-TC06	登录/管理员鉴权；Prisma 持久化；三个目标类型的 KEEP/HIDE/DELETE；注入第二次更新失败并验证事务或补偿	ReportAccept、AdminReview、ContentUpdate、RecordResult
E2E-TC06	普通用户提交举报，管理员查看并处置，随后查询举报状态、目标状态、原因、操作者和时间	Ready → Rejected / Processed

完成标准不是“接口返回 200”，而是终态组合正确、越权请求不改变状态、AI 失败可降级、处置中断后可重试或回滚，并且最终结果能够通过独立查询验证。